**COMPUTER VISION FÜR AUTONOMES FAHREN**

**Hochschule Bochum, WiSe 2023/24**

# VERKEHRZEICHENERKENNUNG UND DEREN KLASSIFIKATION

## Zusammenfassung der Arbeit

Diese Arbeit konzentriert sich auf die Anwendung von Computer-Vision-Techniken im Kontext des autonomen Fahrens, insbesondere auf die Verkehrszeichenerkennung. Ziel ist es, ein neuronales Netzwerk in Python zu implementieren, das Bilder des „German Traffic Sign Recognition Benchmark“ (GTSRB) Datensatzes einliest und verarbeitet. Der Code umfasst Schritte wie das Einlesen der Bilder, die Verarbeitung durch das neuronale Netzwerk, die Bestimmung der Genauigkeit, die Berechnung der Konfusionsmatrix und den Vergleich der Ergebnisse mit der vorhandenen Literatur. Im folgenden werden die Methodik, die Ergebnisse und Schlussfolgerungen dargelegt.

## Einleitung

Die Idee, autonome Fahrzeuge zu entwickeln, begleitet uns bereits seit einiger Zeit. Computer Vision hat dabei eine entscheidende Rolle übernommen. Die Verkehrszeichenerkennung ist ein entscheidender Aspekt, um die Sicherheit und Effizienz im Straßenverkehr zu gewährleisten.

Seit den frühesten Überlegungen zur Automatisierung von Fahrzeugen hat sich die Computer Vision zu einer Schlüsseltechnologie entwickelt. Diese Arbeit konzentriert sich auf die Implementierung eines neuronalen Netzwerks in Python für die Verkehrszeichenerkennung, wobei der „German Traffic Sign Recognition Benchmark “(GTSRB) Datensatz als Grundlage dient.

Die Dokumentation des Codes erfolgt in Deutsch oder Englisch, wodurch die Reproduzierbarkeit der Ergebnisse sichergestellt und das Verständnis für die angewandten Methoden gefördert wird.

Der Vergleich der erzielten Ergebnisse mit der vorhandenen Literatur ermöglicht es, die Effektivität des entwickelten Modells besser zu verstehen.

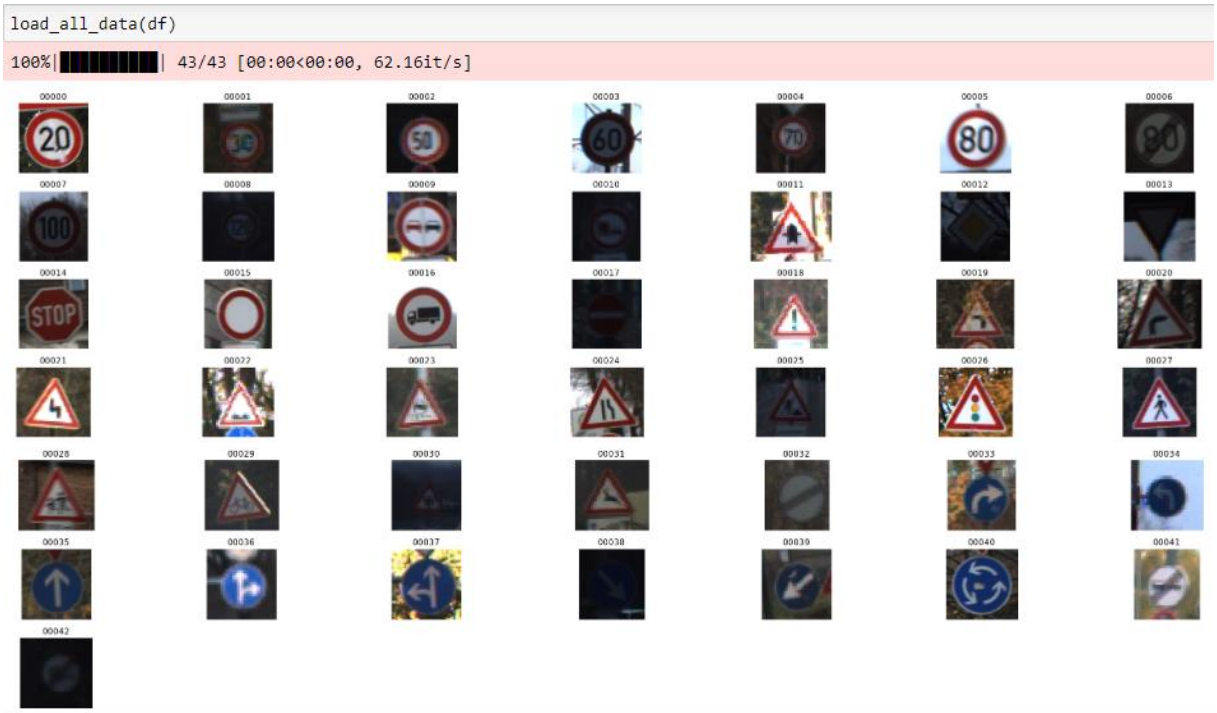
## Datensammlung

Die Datensammlung spielt eine wichtige Rolle bei der Erstellung eines neuronalen Netzes zur Verkehrszeichenerkennung. Der verwendete Datensatz, bereitgestellt durch das „German Traffic Sign Recognition Benchmark “(GTSRB) Projekt der Ruhr-Universität Bochum (RUB), umfasst insgesamt 51.838 Bilder von Verkehrszeichen. Diese umfangreiche Datensammlung ist in 43 verschiedene Klassen von deutschen Verkehrsschildern unterteilt. Die Daten können von der offiziellen Website der Universität (RUB) heruntergeladen werden[1].

Die Daten sind in 43 Klassen organisiert, die die verschiedenen Arten von Verkehrsschildern repräsentieren. Jede Klasse steht für eine bestimmte Kategorie an Verkehrszeichen, von Geschwindigkeitsbegrenzungen bis zu Warnhinweisen. Diese strukturierte Unterteilung ermöglicht ein genaues Training und eine gezielte Evaluierung des neuronalen Netzwerks für die präzise Erkennung verschiedener Verkehrszeichenarten.

[1] https://sid.erda.dk/public/archives/daaeac0d7ce1152aea9b61d9f1e19370/published-archive.html

Die Zusammenstellung der Daten in Klassen gewährleistet eine gezielte Analyse und ermöglicht es dem neuronalen Netzwerk, spezifische Muster für jede Art von Verkehrszeichen zu erlernen. Diese klare Strukturierung trägt dazu bei, die Genauigkeit und Effizienz der Verkehrszeichenerkennung zu verbessern. Durch die Verwendung dieses sorgfältig organisierten GTSRB-Datensatzes der Ruhr-Universität Bochum wird die Qualität der Datenbasis für diese Arbeit sichergestellt.




Übersicht der 43 verschiedenen Verkehrsschilder in Deutschland

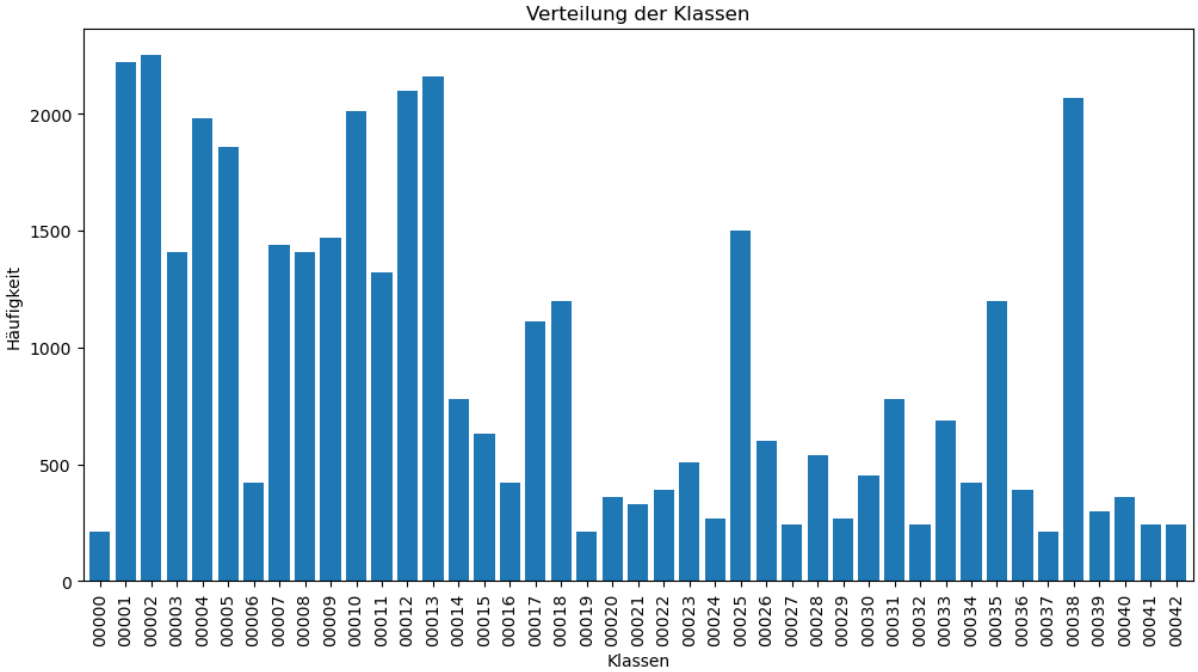
**Datenverteilung**

Die Datenverteilung ist wichtig fürs Training des neuronalen Netzwerks. Die insgesamt 51.838 Bilder, die vom „German Traffic Sign Recognition Benchmark“ (GTSRB) bereitgestellt wurden, sind in zwei Hauptkategorien aufgeteilt: Daten, die für das Training des neuronalen Netzwerks verwendet werden und die, dazu verwendet werden, das neuronale Netz nach dem Training zu evaluieren.

Die Trainingsdaten, die für das Training des neuronalen Netzwerks verwendet werden, umfassen insgesamt 39.209 Bilder. Diese Daten werden sorgfältig auf die 43 Klassen von Verkehrszeichen verteilt, um sicherzustellen, dass das Modell eine breite Palette von Situationen abdecken und zuverlässig Muster und Merkmale lernen kann.

Die Evaluierungs- oder Testdaten, die nach dem Training verwendet werden, umfassen 12.629 Bilder. Diese Daten repräsentieren neue, nicht während des Trainings gesehene Situationen und ermöglichen eine objektive Bewertung der Leistungsfähigkeit des trainierten Modells. Die sorgfältige Aufteilung der Daten in Trainings- und Testsatz ist entscheidend, um sicherzustellen, dass das Modell effektiv auf verschiedene Situationen reagieren kann.

Die folgende Abbildung zeigt eine detaillierte Visualisierung der Verteilung der Klassen in den Trainingsdaten. Diese bietet einen klaren Einblick in die Struktur und den Anteil der verschiedenen Verkehrszeichentypen, die in den Trainingsdaten vertreten sind.




In diesem Diagramm ist eine ungleiche Verteilung der verschiedenen Klassen zu erkennen. Während einige Klassen gut vertreten sind und eine signifikante Anzahl von Bildern aufweisen, gibt es andere Klassen, die wesentlich weniger Bilder enthalten. Beispielsweise enthalten die Klassen 00001, 00002, 00012, 00013 und 00038 jeweils mehr als 2000 Bilder, während Klassen wie 00000, 00019 und 00032 weniger als 500 Bilder aufweisen.

Diese Ungleichheit in der Verteilung der Bilder zwischen den Klassen kann Auswirkungen auf das Training des neuronalen Netzwerks haben. Klassen mit vielen Bildern ermöglichen dem Modell, eine Vielzahl von Situationen zu erleben, was eine robuste Verallgemeinerung fördert. Im Gegensatz dazu bieten Klassen mit weniger Bildern weniger Vielfalt an Beispielen, was die Fähigkeit des Netzwerks zur effektiven Verallgemeinerung während des Trainings einschränken kann.

Ideal wäre es, wenn alle Klassen eine äquivalente Anzahl von Bildern hätten, um eine ausgewogene Repräsentation während des Trainings des Modells zu gewährleisten.

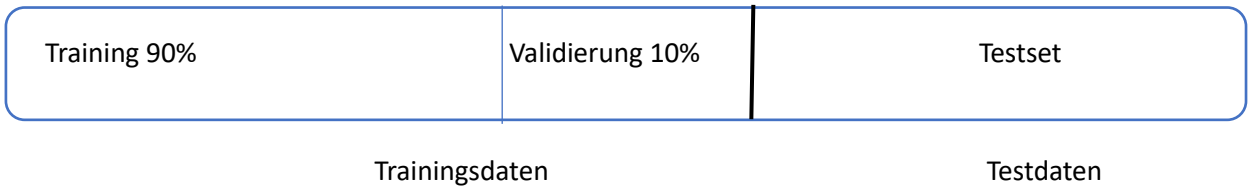
**Aufteilung der Trainingsdaten in Trainings- und Validierungsdaten**

Die Aufteilung der Trainingsdaten in Trainings- und Validierungsdaten ist ein entscheidender Schritt bei der Entwicklung und Evaluierung eines neuronalen Netzwerks. In dieser Aufgabe wird diese Aufteilung mit besonderer Sorgfalt vorgenommen, um sicherzustellen, dass das Modell effektiv trainiert und gleichzeitig zuverlässig validiert wird. Es ist von großer Bedeutung, dass die Trainingsdaten nicht nur für das Training selbst verwendet werden, sondern auch für die Überprüfung der Leistungsfähigkeit des Modells.

In diesem Kontext werden **90%** der Trainingsdaten für das eigentliche Training des neuronalen Netzwerks verwendet. Gleichzeitig wird jedoch auch darauf geachtet, dass ein Teil der Daten separat für die Validierung verwendet wird.

Die restlichen **10%** der Trainingsdaten dienen der Validierung des Modells. Dieser Validierungssatz wird während des Trainingsprozesses eingesetzt, um die Leistungsfähigkeit des Modells auf Daten zu überprüfen, die es während des Trainings noch nicht gesehen hat. Dies ist entscheidend, um

sicherzustellen, dass das Modell nicht nur auswendig lernt, sondern auch in der Lage ist, auf neue, unbekannte Daten effektiv zu reagieren.




**Vorverarbeitung der Daten**

Die Vorverarbeitung der Daten spielt eine wichtige Rolle in der Entwicklung und neuronaler Netzwerke, insbesondere im Kontext der Bildverarbeitung. Ein wesentlicher Aspekt dieses Prozesses besteht in der **Erweiterung** der Trainingsdaten durch verschiedene **Techniken**, um dem neuronalen Netzwerk die Möglichkeit zu bieten, eine breitere Vielfalt von Mustern und Merkmalen zu erlernen.

Die Bedeutung der Datenpräparation liegt darin, dem Modell eine ausreichende Anzahl von Beispielen zu bieten, um eine effektive und generalisierte Verkehrszeichenerkennung zu ermöglichen. Durch die Erweiterung der Trainingsdaten können seltene oder ungewöhnliche Verkehrsschilder besser berücksichtigt werden, was die Robustheit und Zuverlässigkeit des Modells verbessert.

Insbesondere bei der Bildverarbeitung beinhaltet die Vorverarbeitung oft:

**Reskalierung (Normalization):** Um sicherzustellen, dass die Pixelwerte im Bereich zwischen 0 und 1 liegen, was die Konvergenz während des Trainings verbessern kann.

**Rotation, Verschiebung und Verzerrung:** Durch diese Transformationen werden die Trainingsdaten künstlich erweitert. Beispielsweise ermöglichen Rotationen dem Modell, Verkehrszeichen in unterschiedlichen Ausrichtungen zu erkennen.

**Spiegelung (horizontal_flip):** Dies kann die Trainingsdaten weiter diversifizieren, indem sie das Modell auf spiegelverkehrte Verkehrszeichen vorbereiten. Es ist jedoch wichtig, bei der Anwendung der horizontalen Spiegelung Vorsicht walten zu lassen, insbesondere bei Verkehrsschildern. Die Gefahr besteht darin, dass nach der Spiegelung das Ergebnis der Bildklasse möglicherweise die Charakteristika einer anderen Klasse übernehmen könnte.

Die Bildklasse 00033 könnte beispielsweise nach der Spiegelung Merkmale aufweisen, die denen der Klasse 00034 ähneln. Ähnlich könnte die Bildklasse 00036 nach der Spiegelung Eigenschaften aufweisen, die denen der Klasse 00037 ähneln, und so weiter. Diese potenzielle Übernahme von Merkmalen zwischen den Klassen muss bei der Entscheidung für die Anwendung der horizontalen Spiegelung berücksichtigt werden, um sicherzustellen, dass das Training des Modells konsistent und zuverlässig bleibt.

**Batch-Größe**: Ein zusätzlicher wichtiger Aspekt ist die Definition der Batch-Größe. Diese gibt an, wie viele Beispiele gleichzeitig in das Netzwerk eingespeist werden. Eine angemessene Wahl der Batch-Größe beeinflusst die Effizienz des Trainings und die Ressourcennutzung.

Die Untersuchung mit verschiedenen Batch-Größen in Kombination mit den genannten Vorverarbeitungstechniken ist entscheidend, um die beste Konfiguration für das Training zu finden.

Die Datenpräparation bietet dem neuronalen Netzwerk die Möglichkeit, mehr Szenarien und Variationen von Verkehrsschildern zu sehen, als es in den ursprünglichen Daten vorhanden war. Dadurch kann das Modell besser verallgemeinern und auf bisher nicht gesehene Daten reagieren.

Insgesamt ist die Vorverarbeitung der Daten ein entscheidender Schritt, um sicherzustellen, dass ein neuronales Netzwerk effektiv und robust trainiert wird, und es ermöglicht dem Modell, eine breite Vielfalt von Verkehrsszenarien zu erlernen.

```
#Generator und Iterator für Trainingssatz und Validierungssatz

from tensorflow.keras.preprocessing.image import ImageDataGenerator
train_generator = ImageDataGenerator(

    rescale = 1./255, # Normalisierung von Bildern
    rotation_range = 40, #Vergrößerung von Bildern, um eine Überanpassung zu vermeiden
    shear_range = 0.2,
    zoom_range = 0.2,
    #horizontal_flip  = True, #Mit dieser Option, könnte eine Konfusion zwichen manchen Klassen haben.
    fill_mode = 'nearest'

)

val_generator = ImageDataGenerator(rescale = 1./255)

train_iterator = train_generator.flow_from_dataframe(

    train,
    x_col='images',
    y_col='label',
    target_size=(64,64),
    batch_size=512,
    class_mode='categorical'

)
```

```
val_iterator = val_generator.flow_from_dataframe(

    test,
    x_col='images',
    y_col='label',
    target_size=(64,64),
    batch_size=512,
    class_mode='categorical'


)
```

```
Found 35288 validated image filenames belonging to 43 classes.
Found 3921 validated image filenames belonging to 43 classes.
```

Vorverarbeitung der Trainingsdaten

**Art und Architektur des gewählten neuronalen Netzes.**

Die Wahl des, zur Verarbeitung und Klassifizierung von Bilddaten, passenden neuronalen Netzes spielt eine wesentliche Rolle. Im Kontext der Verkehrszeichenerkennung wird für diese Arbeit eine Convolutional-Neural-Network-Architektur (CNN) gewählt. Diese Entscheidung basiert auf der Fähigkeit von CNNs, räumliche Hierarchien von Merkmalen in Bildern zu lernen, was besonders effektiv für Bildklassifikationsaufgaben ist.

Ein Convolutional Neural Network besteht aus Schichten von Convolutional Layern, Pooling Layern und Fully Connected Layern. Die Convolutional Layer extrahieren Merkmale aus den Eingangsbildern, während die Pooling Layer die räumliche Dimension der Daten reduzieren und die Berechnungskosten verringern. Die Fully Connected Layer am Ende des Netzes dienen zur Klassifikation.

Die Abbildung unten veranschaulicht die Struktur des gewählten neuronalen Netzes:

```python
#Erstellung des Modells
from keras import Sequential
from keras.layers import Conv2D, MaxPool2D, Flatten, Dense, Input
from keras import regularizers

input_shape = (64, 64, 3)  # Grösse des Eingangs

model = Sequential()
model.add(Input(shape=input_shape))  # Separate Eingabeebene

model.add(Conv2D(128, (3,3), activation='relu'))
model.add(MaxPool2D((2,2)))
model.add(Conv2D(256, (3,3), activation='relu'))
model.add(MaxPool2D((2,2)))
model.add(Conv2D(512, (3,3), activation='relu'))
model.add(MaxPool2D((2,2)))
model.add(Flatten())
model.add(Dense(512, activation='relu', kernel_regularizer=regularizers.l2(0.003)))
model.add(Dense(43, activation='softmax'))


# Zusammenstellung des Modells mit der angepassten Verlustfunktion für die Mehrklassenklassifizierung
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

**Model: "sequential"**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 62, 62, 128) | 3,584 |
| max_pooling2d (MaxPooling2D) | (None, 31, 31, 128) | 0 |
| conv2d_1 (Conv2D) | (None, 29, 29, 256) | 295,168 |
| max_pooling2d_1 (MaxPooling2D) | (None, 14, 14, 256) | 0 |
| conv2d_2 (Conv2D) | (None, 12, 12, 512) | 1,180,160 |
| max_pooling2d_2 (MaxPooling2D) | (None, 6, 6, 512) | 0 |
| flatten (Flatten) | (None, 18432) | 0 |
| dense (Dense) | (None, 512) | 9,437,696 |
| dense_1 (Dense) | (None, 43) | 22,059 |

**Total params:** 10,938,667 (41.73 MB)
**Trainable params:** 10,938,667 (41.73 MB)
**Non-trainable params:** 0 (0.00 B)

Die CNN-Architektur ermöglicht es dem Netzwerk, lokale Merkmale wie Kanten und Texturen zu erlernen und diese dann zu kombinieren, um komplexe Muster und Strukturen zu erkennen. Das ist besonders wichtig für die Verkehrszeichenerkennung, da verschiedene Verkehrsschilder unterschiedliche visuelle Merkmale aufweisen.

Die Auswahl der Architektur und die Feinabstimmung der Hyperparameter des neuronalen Netzes sind entscheidend für die Leistungsfähigkeit des Modells. Durch das Training mit den vorverarbeiteten Daten und der sorgfältigen Definition der Netzwerkarchitektur kann ein präzises und zuverlässiges Modell für die Verkehrszeichenerkennung entwickelt werden.

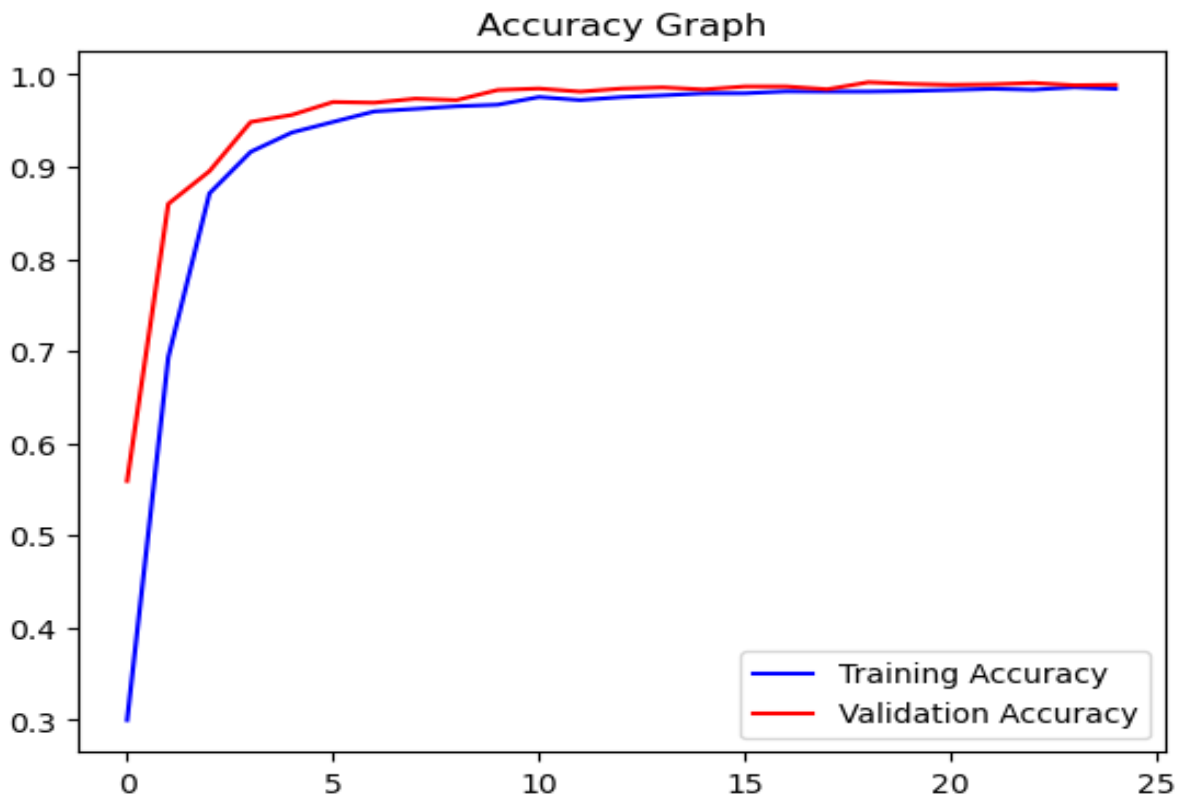
**Trainingsphase**

Die Abbildung unten zeigt den Verlauf des Trainings des Netzwerks über 25 Epochen. Sie visualisiert die Fortschritte der Trainings- und Validierungskostenfunktionen sowie die Genauigkeit nach jeder Epoche. Dieser Verlauf ermöglicht eine detaillierte Analyse der Lernentwicklung des Modells über die Trainingszeit hinweg.

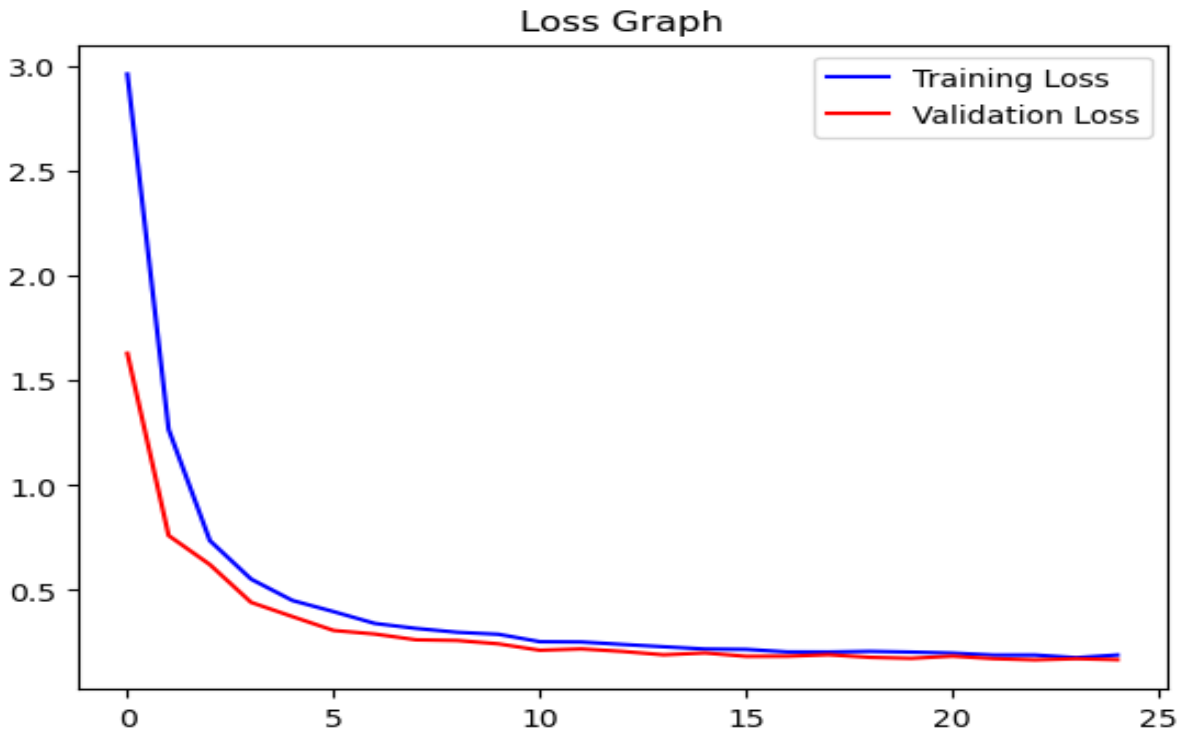
```
# Trainieren des Modells
history = model.fit(train_iterator, epochs=25, validation_data=val_iterator, verbose=1)

#Anzeigen der Grafik
plot_graph(history)
```

```
69/69 ━━━━━━━━━━━━━━━━━━━━ 498s 7s/step - accuracy: 0.9803 - loss: 0.2154 - val_accuracy: 0.9870 - val_loss: 0.1840
Epoch 17/25
69/69 ━━━━━━━━━━━━━━━━━━━━ 497s 7s/step - accuracy: 0.9828 - loss: 0.2032 - val_accuracy: 0.9870 - val_loss: 0.1847
Epoch 18/25
69/69 ━━━━━━━━━━━━━━━━━━━━ 502s 7s/step - accuracy: 0.9818 - loss: 0.2055 - val_accuracy: 0.9839 - val_loss: 0.1916
Epoch 19/25
69/69 ━━━━━━━━━━━━━━━━━━━━ 500s 7s/step - accuracy: 0.9819 - loss: 0.2048 - val_accuracy: 0.9916 - val_loss: 0.1791
Epoch 20/25
69/69 ━━━━━━━━━━━━━━━━━━━━ 501s 7s/step - accuracy: 0.9828 - loss: 0.2032 - val_accuracy: 0.9898 - val_loss: 0.1744
Epoch 21/25
69/69 ━━━━━━━━━━━━━━━━━━━━ 498s 7s/step - accuracy: 0.9838 - loss: 0.2006 - val_accuracy: 0.9888 - val_loss: 0.1849
Epoch 22/25
69/69 ━━━━━━━━━━━━━━━━━━━━ 499s 7s/step - accuracy: 0.9851 - loss: 0.1919 - val_accuracy: 0.9893 - val_loss: 0.1728
Epoch 23/25
69/69 ━━━━━━━━━━━━━━━━━━━━ 510s 7s/step - accuracy: 0.9835 - loss: 0.1900 - val_accuracy: 0.9908 - val_loss: 0.1670
Epoch 24/25
69/69 ━━━━━━━━━━━━━━━━━━━━ 496s 7s/step - accuracy: 0.9888 - loss: 0.1733 - val_accuracy: 0.9883 - val_loss: 0.1728
Epoch 25/25
69/69 ━━━━━━━━━━━━━━━━━━━━ 497s 7s/step - accuracy: 0.9864 - loss: 0.1854 - val_accuracy: 0.9888 - val_loss: 0.1678
```




Auf der obigen Abbildung ist eine Annäherung zwischen dem Trainings- und Validierungsgraphen zu erkennen. Dies bedeutet, dass das neuronale Netzwerk die während des Trainings auf den Validierungsdaten erworbenen Erkenntnisse gut nutzt. Dieses Phänomen zeigt, dass das Modell in der Lage ist, effektiv auf den Validierungsdaten zu generalisieren, was ein gutes Anzeichen für seine Leistungsfähigkeit ist.



Hier zeigt sich eine Konvergenz der Kostenfunktionen gegen null, was wünschenswert ist. Dies deutet, dass der Klassifikationsfehler signifikant abnimmt. Diese Konvergenz ist ein positives Zeichen dafür, dass das Modell erfolgreich trainiert wurde und gut auf die Daten passt.

**Auswertung des Modells mit Echt- oder Testdaten (Vorhersagen, Berechnung der Genauigkeit und der Konfusionsmatrix)**

Die Auswertung des Modells mit Echt- oder Testdaten ist ein kritischer Schritt, um die Leistungsfähigkeit des trainierten neuronalen Netzwerks zu beurteilen. Nachdem das Modell erfolgreich trainiert wurde, ist es an der Zeit, es mit Daten zu evaluieren, die es während des Trainings nicht gesehen hat. Diese Evaluierung umfasst verschiedene Schritte:

Vorhersagen: Das trainierte Modell wird verwendet, um Vorhersagen für die Echt- oder Testdaten zu generieren. Jedes Verkehrszeichen in den Testdaten wird vom Modell klassifiziert, und die Vorhersagen werden extrahiert.

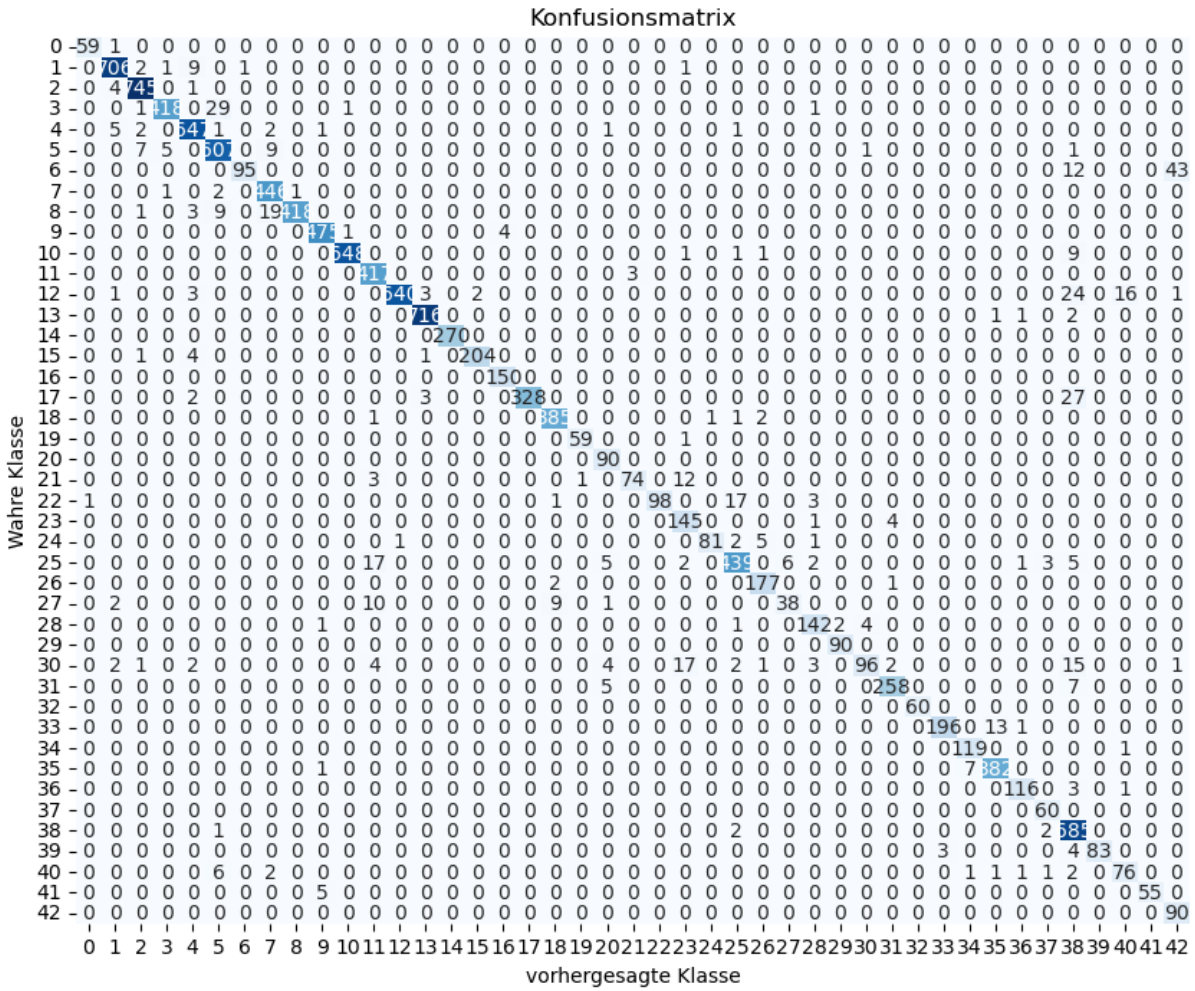
**Berechnung der Genauigkeit**: Die Genauigkeit des Modells wird berechnet, indem die Anzahl der korrekten Vorhersagen durch die Gesamtanzahl der Vorhersagen geteilt wird. Dies gibt Aufschluss darüber, wie gut das Modell auf neuen, nicht trainierten Daten abschneidet.

**Berechnung der Konfusionsmatrix**: Die Konfusionsmatrix ist ein wichtiges Instrument zur Bewertung der Klassifikationsleistung. Sie zeigt, wie viele Instanzen jeder Klasse korrekt oder inkorrekt klassifiziert wurden. Dies ermöglicht eine detaillierte Analyse der Stärken und Schwächen des Modells in Bezug auf verschiedene Verkehrsschilderklassen.

Unten steht eine detaillierte Visualisierung dieser Auswertungsschritte:

```
Prediction:   100%|██████████ |12630/12630 [00:00<00:00, 115273.13it/s]

Image 1 - Vorhersagen : 16, Wahre Klasse : 16, Vertrautheit : 1.00
Image 2 - Vorhersagen : 1, Wahre Klasse : 1, Vertrautheit : 1.00
Image 3 - Vorhersagen : 38, Wahre Klasse : 38, Vertrautheit : 1.00
Image 4 - Vorhersagen : 33, Wahre Klasse : 33, Vertrautheit : 0.99
Image 5 - Vorhersagen : 11, Wahre Klasse : 11, Vertrautheit : 1.00
Image 6 - Vorhersagen : 38, Wahre Klasse : 38, Vertrautheit : 1.00
Image 7 - Vorhersagen : 18, Wahre Klasse : 18, Vertrautheit : 1.00
Image 8 - Vorhersagen : 12, Wahre Klasse : 12, Vertrautheit : 0.99
Image 9 - Vorhersagen : 25, Wahre Klasse : 25, Vertrautheit : 1.00
Image 10 - Vorhersagen : 35, Wahre Klasse : 35, Vertrautheit : 1.00
Image 11 - Vorhersagen : 12, Wahre Klasse : 12, Vertrautheit : 0.96
Image 12 - Vorhersagen : 7, Wahre Klasse : 7, Vertrautheit : 1.00
Image 13 - Vorhersagen : 23, Wahre Klasse : 23, Vertrautheit : 0.99
Image 14 - Vorhersagen : 7, Wahre Klasse : 7, Vertrautheit : 0.74
Image 15 - Vorhersagen : 4, Wahre Klasse : 4, Vertrautheit : 1.00
Image 16 - Vorhersagen : 9, Wahre Klasse : 9, Vertrautheit : 1.00
...   ...      ...     ...   ...   ...    ...      ...        ...
...   ...      ...     ...   ...   ...    ...      ...        ...
Image 12628 - Vorhersagen : 42, Wahre Klasse : 6, Vertrautheit : 0.69
Image 12629 - Vorhersagen : 7, Wahre Klasse : 7, Vertrautheit : 1.00
Image 12630 - Vorhersagen : 10, Wahre Klasse : 10, Vertrautheit : 1.00
```




***Genauigkeit des Modells auf dem Testdatensatz: 95.67%***

Diese Evaluierung ist kritisch, um sicherzustellen, dass das trainierte Modell in der Lage ist, zuverlässige Vorhersagen für neue Daten zu treffen. Die visuelle Darstellung ermöglicht es, die Ergebnisse intuitiv zu interpretieren und gegebenenfalls Anpassungen am Modell oder der Datenvorverarbeitung vorzunehmen, um die Leistung zu verbessern.

**Misklassifizierte Klassen und die Gründe dafür**

Nach der Auswertung des Netzwerks mit Testdaten lässt sich weiterhin feststellen, dass es zu Verwechslungen zwischen manchen Klassen kommt. Ein Beispiel dafür ist die Klasse 00042, die 43-mal fälschlicherweise als Klasse 00006 vorhergesagt wurde und die Klasse 00005, die auch 29-mal als Klasse 00003 vorhergesagt wurde. Dies ist wahrscheinlich darauf zurückzuführen, dass es in diesen Klassen nicht genügend Bilder gibt. Während des Trainings hat das Netzwerk möglicherweise nicht genügend verschiedene Situationen in diesen Klassen erlebt, was seine Fähigkeit beeinträchtigen kann, mehr Merkmale zu extrahieren und somit die Genauigkeit der Vorhersagen zu erhöhen.

**Vergleich der Ergebnisse mit der vorhandenen Literatur**

Das neuronale Netzwerk, hat im Rahmen dieser Arbeit herausragende Leistungen gezeigt und kann erfolgreich mit den Ergebnissen der von der Ruhr Universität Bochum durchgeführten Klassifizierung verglichen werden. Mit einer beachtlichen Genauigkeit von **95,67%** hat das Modell gezeigt, dass es in der Klassifizierung von Verkehrsschildern überzeugend abschneidet.

Im Vergleich zu den Ergebnissen der Klassifizierung durch RUB, sitzt dieses Modell auf dem siebten Platz, was nicht allzu schlecht ist. Hier sind die von der RUB erstellten Ergebnisse:

| CCR (%) | Team | Method |
|---|---|---|
| **99,46%** | **IDSIA (CNNs)** | Committee of CNNs |
| **99,22%:** | **INI-RTCV** | Human (best individual) |
| **98,84%:** | **INI-RTCV** | Human (average) |
| **98,31%:** | **Sermanet** | Multi-Scale CNN |
| **96,14%:** | **CAOR** | Random Forests |
| **95,68%:** | **INI-RTCV** | LDA (HOG 2) |
| **93,18%:** | **INI-RTCV** | LDA (HOG 1) |
| **92,34%:** | **INI-RTCV** | LDA (HOG 3) |

**Zusammenfassung**

In dieser Arbeit wurde ein neuronales Netzwerk für die Klassifikation von Bildern implementiert und trainiert. Die Analyse des Trainingsverlaufs sowie der Evaluierung mit Testdaten ermöglichte Einblicke in die Leistungsfähigkeit des Modells. Mit einer beeindruckenden Genauigkeit von **95,67 %** zeigt das Modell eine starke Fähigkeit zur korrekten Klassifikation der Testdaten.

Es wurde festgestellt, dass es eine ungleiche Verteilung von Bildern zwischen den Klassen gibt, was sich auf die Fähigkeit des Netzwerks zur Generalisierung auswirken kann. Des Weiteren wurden misklassifizierte Klassen identifiziert, insbesondere in Klassen mit begrenzter Datenvielfalt.

Trotz dieser Herausforderungen konnte das Modell eine Konvergenz der Kostenfunktionen gegen null erreichen, was auf eine erfolgreiche Reduzierung des Klassifikationsfehlers hinweist. Die Annäherung zwischen den Trainings- und Validierungsgraphen zeigt eine effektive Nutzung der während des Trainings erworbenen Kenntnisse auf den Validierungsdaten.

Jedoch wurden Schwächen bei der Vorhersage von Klassen mit begrenzten Datenmengen festgestellt, was auf die Bedeutung eines ausgewogenen Datensatzes für das Training hinweist. In Zukunft könnte die Leistung des Modells durch den Einsatz von Techniken wie Data Augmentation oder Transfer Learning weiter verbessert werden.

Abschließend unterstreicht diese Semesterabschlussarbeit die entscheidende Bedeutung einer sorgfältigen Datenanalyse, einer ausgewogenen Datenverteilung und einer kontinuierlichen Modellverbesserung für den Erfolg von Bildklassifikationsaufgaben im Bereich des maschinellen Lernens. Diese Aspekte sind nicht nur grundlegend für das Verständnis und die Optimierung von Bildklassifikationsmodellen, sondern spielen auch eine wesentliche Rolle in wegweisenden Technologiebereichen wie dem autonomen Fahren.

Im Kontext der Entwicklung autonomer Fahrsysteme nimmt die Computer Vision einen bedeutenden Platz ein. Die Fähigkeit autonomer Fahrzeuge, ihre Umgebung zu interpretieren und darauf zu reagieren, beruht weitgehend auf leistungsfähigen Modellen der Bildverarbeitung. Daher können die Erkenntnisse aus dieser Datenanalyse und den Modellierungsmethoden auf sinnvolle Weise im vielversprechenden Bereich des autonomen Fahrens angewendet werden. Durch die Kombination einer gründlichen Datenanalyse mit fortschrittlichen Techniken der Computer Vision tragen wir zur Schaffung effizienterer und sichererer Lösungen für die Zukunft der Mobilität bei.